

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)**

Department of Artificial Intelligence and Data Science

DSA0503 – Query Processing For Data Science

Lab Experiments 1-10

Name: Renee
Vincent ASR

**Register
Number:**192424161

1. Write a Pandas program to select the distinct department id from employees file.

Select Distinct Department IDs

Aim:

To select the distinct department IDs from the departments dataset using Pandas.

Algorithm:

1. Import the Pandas library.
2. Read the departments.csv file.
3. Extract unique values from the DEPARTMENT_ID column.
4. Display the distinct department IDs.

Input CSV File: employees.csv

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
30	Purchasing	114	1700
40	Human Resources	203	2400
50	Shipping	121	1500
60	IT	103	1400
70	Public Relations	204	2700
80	Sales	145	2500
90	Executive	100	1700
100	Finance	108	1700
110	Accounting	205	1700
120	Treasury	0	1700
130	Corporate Tax	0	1700
140	Control And Credit	0	1700
150	Shareholder Services	0	1700
160	Benefits	0	1700
170	Manufacturing	0	1700
180	Construction	0	1700
190	Contracting	0	1700
200	Operations	0	1700
210	IT Support	0	1700
220	NOC	0	1700
230	IT Helpdesk	0	1700

240	Government Sales	0	1700
250	Retail Sales	0	1700
260	Recruiting	0	1700
270	Payroll	0	1700

Code:

```
import pandas as pd

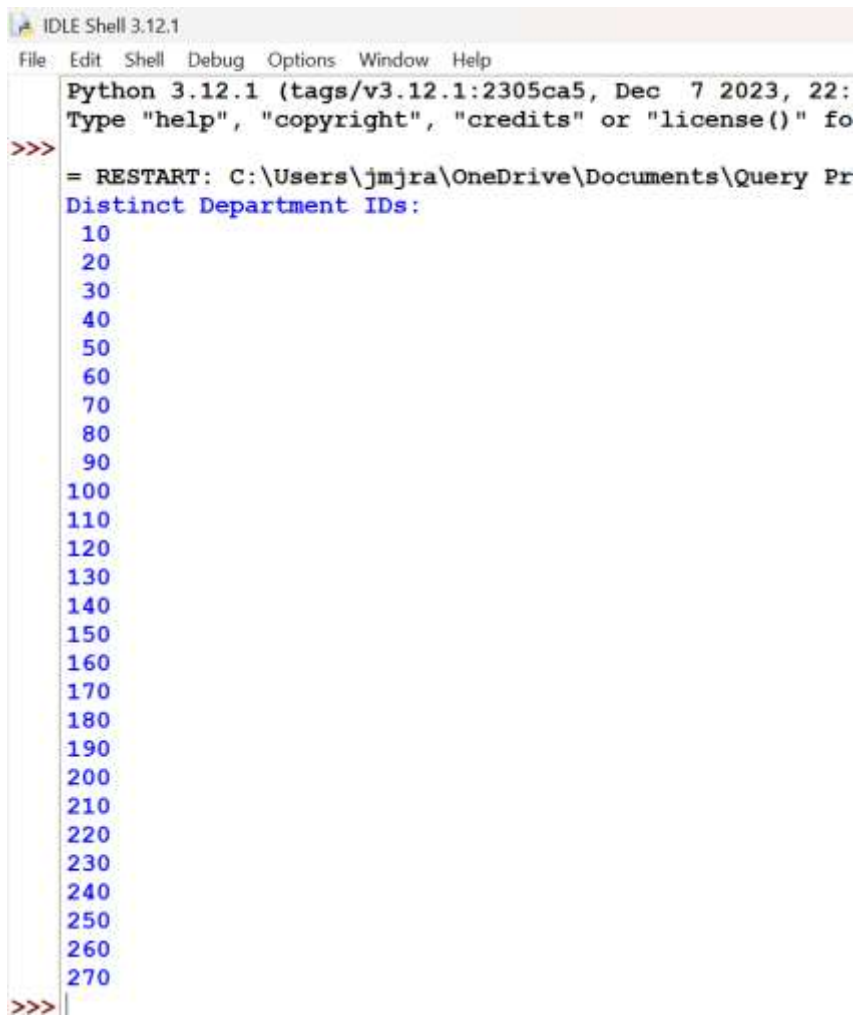
df = pd.read_csv("employees.csv")

distinct_department_ids = df["DEPARTMENT_ID"].drop_duplicates()

print("Distinct Department IDs:")

print(distinct_department_ids.to_string(index=False))
```

Output:



```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:
Type "help", "copyright", "credits" or "license()" fo
>>>
= RESTART: C:\Users\jnjra\OneDrive\Documents\Query Pr
Distinct Department IDs:
10
20
30
40
50
60
70
80
90
100
110
120
130
140
150
160
170
180
190
200
210
220
230
240
250
260
270
>>>
```

Result:

Distinct department IDs were displayed successfully.

2. Write a Pandas program to display the IDs of employees who have done two or more jobs in the past.

Display Employees Who Have Done Two or More Jobs

Aim:

To identify employees who have worked in two or more jobs.

Algorithm:

1. Import the Pandas library.
2. Read the job_history.csv file.
3. Count job records for each employee.
4. Display employee IDs having two or more jobs.

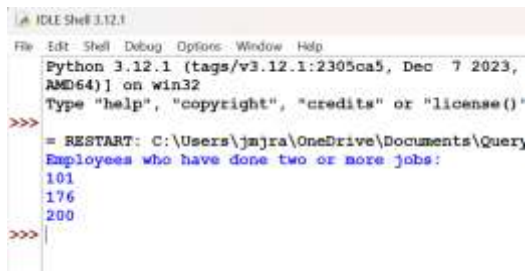
Input CSV File: job_history.csv

```
EMPLOYEE_ID,START_DATE,END_DATE,JOB_ID,DEPARTMENT_ID
102,2001-01-13,2006-07-24,IT_PROG,60
101,1997-09-21,2001-10-27,AC_ACCOUNT,110
101,2001-10-28,2005-03-15,AC_MGR,110
201,2004-02-17,2007-12-19,MK_REP,20
114,2006-03-24,2007-12-31,ST_CLERK,50
122,2007-01-01,2007-12-31,ST_CLERK,50
200,1995-09-17,2001-06-17,AD_ASST,90
176,2006-03-24,2006-12-31,SA_REP,80
176,2007-01-01,2007-12-31,SA_MAN,80
200,2002-07-01,2006-12-31,AC_ACCOUNT,90
```

Code:

```
import pandas as pd
df = pd.read_csv("job_history.csv")
job_count = df.groupby("EMPLOYEE_ID").size()
employees = job_count[job_count >= 2].index
print("Employees who have done two or more jobs:")
for emp_id in employees:
    print(emp_id)
```

Output:

A screenshot of a Python IDLE Shell window. The title bar says 'IDLE Shell 3.12.1'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The shell shows the Python version 'Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, AMD64)' on win32. It displays a query result for 'Employees who have done two or more jobs:' with three rows of data: 101, 176, and 200. The prompt '>>>' is visible at the bottom.

```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023,
AMD64) on win32
Type "help", "copyright", "credits" or "license()"
>>>
= RESTART: C:\Users\jmjra\OneDrive\Documents\Quer
Employees who have done two or more jobs:
101
176
200
>>>
```

Result:

Employees with two or more jobs were displayed successfully.

3. Write a Pandas program to display the details of jobs in descending sequence on job title.

Display Job Details in Descending Order of Job Title

Aim:

To sort and display job details in descending order of job title.

Algorithm:

1. Import the Pandas library.
2. Read the jobs.csv file.
3. Sort the data by JOB_TITLE in descending order.
4. Display the sorted job details.

Input CSV File: jobs.csv

JOB_ID,JOB_TITLE,MIN_SALARY,MAX_SALARY

AD_PRES,President,20080,40000

AD_VP,Administration Vice President,15000,30000

AD_ASST,Administration Assistant,3000,6000

FI_MGR,Finance Manager,8200,16000

FI_ACCOUNT,Accountant,4200,9000

AC_MGR,Accounting Manager,8200,16000

AC_ACCOUNT,Public Accountant,4200,9000

SA_MAN,Sales Manager,10000,20080

SA_REP,Sales Representative,6000,12008

PU_MAN,Purchasing Manager,8000,15000

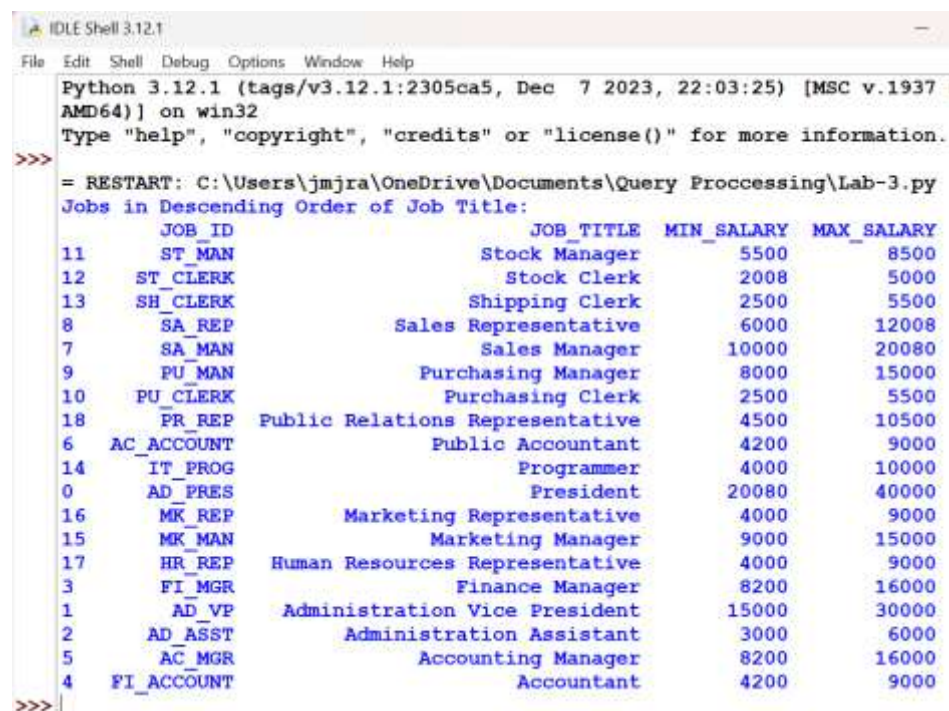
PU_CLERK,Purchasing Clerk,2500,5500

ST_MAN,Stock Manager,5500,8500
 ST_CLERK,Stock Clerk,2008,5000
 SH_CLERK,Shipping Clerk,2500,5500
 IT_PROG,Programmer,4000,10000
 MK_MAN,Marketing Manager,9000,15000
 MK_REP,Marketing Representative,4000,9000
 HR_REP,Human Resources Representative,4000,9000
 PR_REP,Public Relations Representative,4500,10500

Code:

```
import pandas as pd
df = pd.read_csv("jobs.csv")
sorted_jobs = df.sort_values(by="JOB_TITLE", ascending=False)
print("Jobs in Descending Order of Job Title:")
print(sorted_jobs)
```

Output:



```
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 i
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\jnjra\OneDrive\Documents\Query Processing\Lab-3.py
Jobs in Descending Order of Job Title:
  JOB_ID  JOB_TITLE  MIN_SALARY  MAX_SALARY
11  ST_MAN  Stock Manager      5500      8500
12  ST_CLERK  Stock Clerk       2008       5000
13  SH_CLERK  Shipping Clerk     2500       5500
8   SA_REP   Sales Representative     6000     12008
7   SA_MAN   Sales Manager      10000     20080
9   PU_MAN   Purchasing Manager     8000     15000
10  PU_CLERK  Purchasing Clerk      2500       5500
18  PR_REP   Public Relations Representative  4500     10500
6   AC_ACCOUNT  Public Accountant      4200       9000
14  IT_PROG   Programmer           4000     10000
0   AD_PRES   President            20080     40000
16  MK_REP   Marketing Representative     4000       9000
15  MK_MAN   Marketing Manager       9000     15000
17  HR_REP   Human Resources Representative  4000       9000
3   FI_MGR   Finance Manager        8200     16000
1   AD_VP    Administration Vice President  15000     30000
2   AD_ASST  Administration Assistant      3000       6000
5   AC_MGR   Accounting Manager       8200     16000
4   FI_ACCOUNT  Accountant           4200       9000
>>>
```

Result:

Job details were displayed in descending order successfully.

4. Write a Pandas program to create a line plot of the historical stock prices of Alphabet Inc. between two specific dates.

Create a Line Plot of Alphabet Stock Prices

Aim:

To create a line plot of Alphabet Inc. historical stock prices between two dates.

Algorithm:

1. Import Pandas and Matplotlib.
2. Read the alphabet_stock_data.csv file.
3. Filter the records between the specified dates.
4. Plot the closing stock prices as a line graph.

Input CSV File: alphabet_stock_data.csv

Date,Open,High,Low,Close,Volume

2024-01-01,140.5,142.0,139.8,141.2,1200000

2024-01-02,141.2,143.1,140.5,142.8,1350000

2024-01-03,142.8,144.0,141.9,143.5,1400000

2024-01-04,143.5,145.2,142.7,144.8,1500000

2024-01-05,144.8,146.3,144.0,145.6,1600000

2024-01-08,145.6,147.0,145.2,146.4,1580000

2024-01-09,146.4,148.1,145.9,147.5,1700000

2024-01-10,147.5,149.0,147.0,148.2,1750000

2024-01-11,148.2,150.0,147.8,149.6,1800000

2024-01-12,149.6,151.2,149.0,150.8,1850000

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("Alphabet_Stock.csv")
df["Date"] = pd.to_datetime(df["Date"])
start_date = "2020-01-01"
end_date = "2020-12-31"
filtered_data = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]
plt.figure(figsize=(10,5))
```

```
plt.plot(filtered_data["Date"], filtered_data["Close"], marker='o')
plt.title("Alphabet Inc. Historical Stock Prices")
plt.xlabel("Date")
plt.ylabel("Closing Price (USD)")
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

Output:



Result:

The line plot of Alphabet stock prices was generated successfully.

5. Write a Pandas program to create a bar plot of the trading volume of Alphabet Inc. stock between two specific dates.

Create a Bar Plot of Alphabet Trading Volume

Aim:

To create a bar plot of Alphabet Inc. trading volume between two dates.

Algorithm:

1. Import Pandas and Matplotlib.
2. Read the alphabet_stock_data.csv file.
3. Filter the required date range.
4. Plot the trading volume as a bar chart.

Input CSV File: alphabet_stock_data.csv

Date,Open,High,Low,Close,Volume

2024-01-01,140.5,142.0,139.8,141.2,1200000

2024-01-02,141.2,143.1,140.5,142.8,1350000

2024-01-03,142.8,144.0,141.9,143.5,1400000

2024-01-04,143.5,145.2,142.7,144.8,1500000

2024-01-05,144.8,146.3,144.0,145.6,1600000

2024-01-08,145.6,147.0,145.2,146.4,1580000

2024-01-09,146.4,148.1,145.9,147.5,1700000

2024-01-10,147.5,149.0,147.0,148.2,1750000

2024-01-11,148.2,150.0,147.8,149.6,1800000

2024-01-12,149.6,151.2,149.0,150.8,1850000

Code:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("Alphabet_Stock.csv")
```

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
start_date = "2020-01-01"
```

```
end_date = "2020-12-31"
```

```
filtered_data = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]
```

```
plt.figure(figsize=(10,5))
```

```
plt.bar(filtered_data["Date"], filtered_data["Volume"])
```

```
plt.title("Alphabet Inc. Trading Volume")
```

```
plt.xlabel("Date")
```

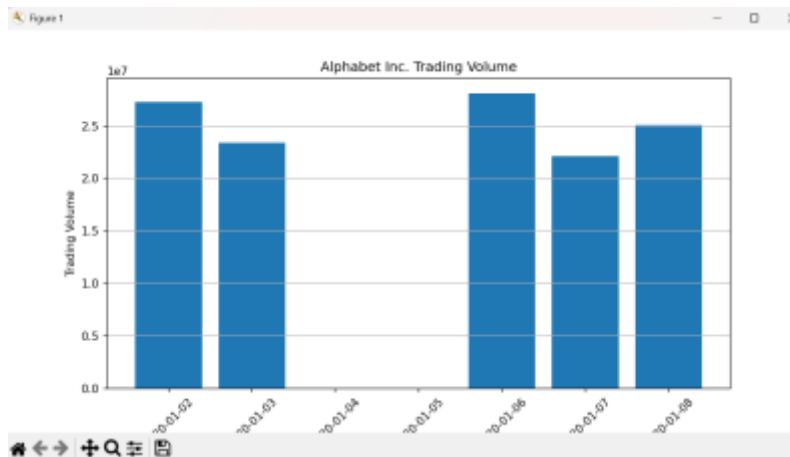
```
plt.ylabel("Trading Volume")
```

```
plt.xticks(rotation=45)
```

```
plt.grid(axis='y')
```

```
plt.show()
```


Output:



Result:

The trading volume bar plot was generated successfully.

6. Write a Pandas program to create a scatter plot of the trading volume/stock prices of Alphabet Inc. stock between two specific dates.

Create a Scatter Plot of Alphabet Stock Prices/Trading Volume

Aim:

To create a scatter plot showing the relationship between stock price and trading volume.

Algorithm:

1. Import Pandas and Matplotlib.
2. Read the alphabet_stock_data.csv file.
3. Select the Close Price and Volume columns.
4. Plot a scatter graph.

Input CSV File: Alpha_Stock.csv

Date,Open,High,Low,Close,Volume

2024-01-01,140.5,142.0,139.8,141.2,1200000

2024-01-02,141.2,143.1,140.5,142.8,1350000

2024-01-03,142.8,144.0,141.9,143.5,1400000

2024-01-04,143.5,145.2,142.7,144.8,1500000

2024-01-05,144.8,146.3,144.0,145.6,1600000

2024-01-08,145.6,147.0,145.2,146.4,1580000

2024-01-09,146.4,148.1,145.9,147.5,1700000

2024-01-10,147.5,149.0,147.0,148.2,1750000

2024-01-11,148.2,150.0,147.8,149.6,1800000

2024-01-12,149.6,151.2,149.0,150.8,1850000

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

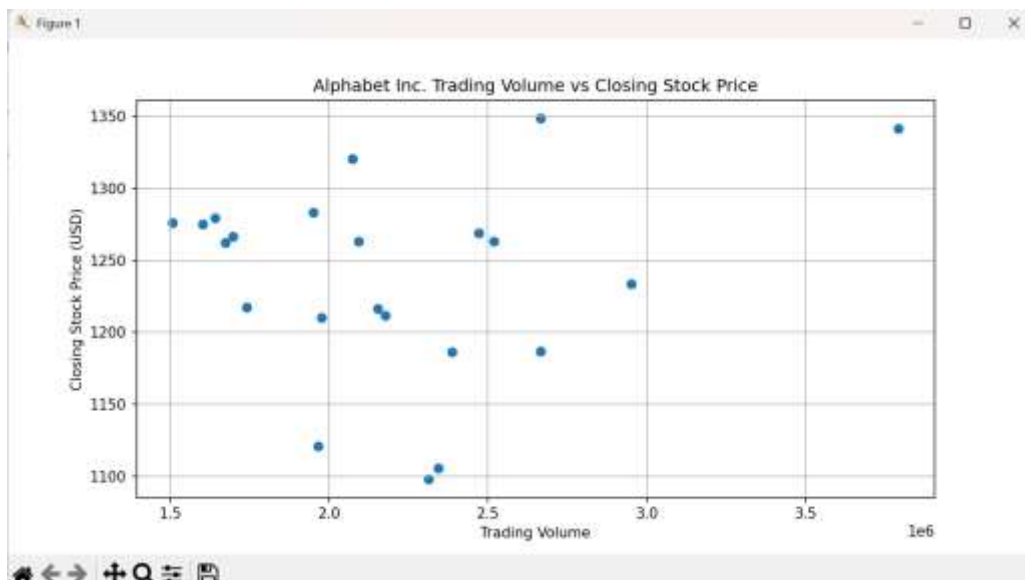
df = pd.read_csv("Alpha_Stock.csv")
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)

start_date = "2020-04-01"
end_date = "2020-05-01"

filtered_data = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]

plt.figure(figsize=(10,5))
plt.scatter(filtered_data["Volume"], filtered_data["Close"])
plt.title("Alphabet Inc. Trading Volume vs Closing Stock Price")
plt.xlabel("Trading Volume")
plt.ylabel("Closing Stock Price (USD)")
plt.grid(True)
plt.show()
```

Output:



Result:

The scatter plot of stock prices and trading volume was generated successfully.

7. Write a Pandas program to create a Pivot table and find the maximum and minimum sale value of the items.

Create a Pivot Table to Find Maximum and Minimum Sale Value

Aim:

To create a Pivot Table to find the maximum and minimum sale value of items.

Algorithm:

1. Import the Pandas library.
2. Read the sales_data.csv file.
3. Create a Pivot Table using the Item column.
4. Display the maximum and minimum sale values.

Input CSV File: sales_data.csv

OrderDate,Region,Manager,SalesMan,Item,Units,Unit_price,Sale_amt

1-6-18,East,Martha,Alexander,Television,95,1198,113810
1-23-18,Central,Hermann,Shelli,Home Theater,50,500,25000
2-9-18,Central,Hermann,Luis,Television,36,1198,43128
2-26-18,Central,Timothy,David,Cell Phone,27,225,6075
3-15-18,West,Timothy,Stephen,Television,56,1198,67088
4-1-18,East,Martha,Alexander,Home Theater,60,500,30000
4-18-18,Central,Martha,Steven,Television,75,1198,89850
5-5-18,Central,Hermann,Luis,Television,90,1198,107820
5-22-18,West,Douglas,Michael,Television,32,1198,38336
6-8-18,East,Martha,Alexander,Home Theater,60,500,30000
6-25-18,Central,Hermann,Sigal,Television,90,1198,107820
7-12-18,East,Martha,Diana,Home Theater,29,500,14500
7-29-18,East,Douglas,Karen,Home Theater,81,500,40500
8-15-18,East,Martha,Alexander,Television,35,1198,41930
9-1-18,Central,Douglas,John,Desk,2,125,250
9-18-18,East,Martha,Alexander,Video Games,16,58.5,936
10-5-18,Central,Hermann,Sigal,Home Theater,28,500,14000
10-22-18,East,Martha,Alexander,Cell Phone,64,225,14400

Code:

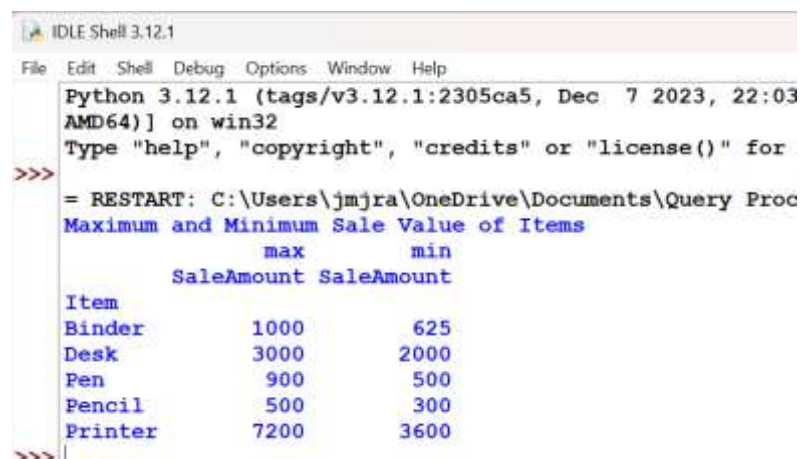
```
import pandas as pd

df = pd.read_csv("sales_data.csv")

pivot = pd.pivot_table(
    df,
    values="SaleAmount",
    index="Item",
    aggfunc=["max", "min"]
)

print("Maximum and Minimum Sale Value of Items")

print(pivot)
```

Output:

```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03
AMD64) on win32
Type "help", "copyright", "credits" or "license()" for
>>>
= RESTART: C:\Users\jmjra\OneDrive\Documents\Query Proc
Maximum and Minimum Sale Value of Items
      max      min
Item  SaleAmount SaleAmount
Binder      1000      625
Desk       3000     2000
Pen         900      500
Pencil      500      300
Printer    7200     3600
>>>
```

Result:

Maximum and minimum sale values were displayed successfully.

8. Write a Pandas program to create a Pivot table and find the item wise unit sold.**Create a Pivot Table to Find Item-wise Units Sold****Aim:**

To create a Pivot Table to find the total units sold for each item.

Algorithm:

1. Import the Pandas library.
2. Read the sales_data.csv file.
3. Create a Pivot Table using the Item column.

4. Display the total units sold.

Input CSV File: sales_data.csv

OrderDate,Region,Manager,SalesMan,Item,Units,Unit_price,Sale_amt

1-6-18,East,Martha,Alexander,Television,95,1198,113810
1-23-18,Central,Hermann,Shelli,Home Theater,50,500,25000
2-9-18,Central,Hermann,Luis,Television,36,1198,43128
2-26-18,Central,Timothy,David,Cell Phone,27,225,6075
3-15-18,West,Timothy,Stephen,Television,56,1198,67088
4-1-18,East,Martha,Alexander,Home Theater,60,500,30000
4-18-18,Central,Martha,Steven,Television,75,1198,89850
5-5-18,Central,Hermann,Luis,Television,90,1198,107820
5-22-18,West,Douglas,Michael,Television,32,1198,38336
6-8-18,East,Martha,Alexander,Home Theater,60,500,30000
6-25-18,Central,Hermann,Sigal,Television,90,1198,107820
7-12-18,East,Martha,Diana,Home Theater,29,500,14500
7-29-18,East,Douglas,Karen,Home Theater,81,500,40500
8-15-18,East,Martha,Alexander,Television,35,1198,41930
9-1-18,Central,Douglas,John,Desk,2,125,250
9-18-18,East,Martha,Alexander,Video Games,16,58.5,936
10-5-18,Central,Hermann,Sigal,Home Theater,28,500,14000
10-22-18,East,Martha,Alexander,Cell Phone,64,225,14400

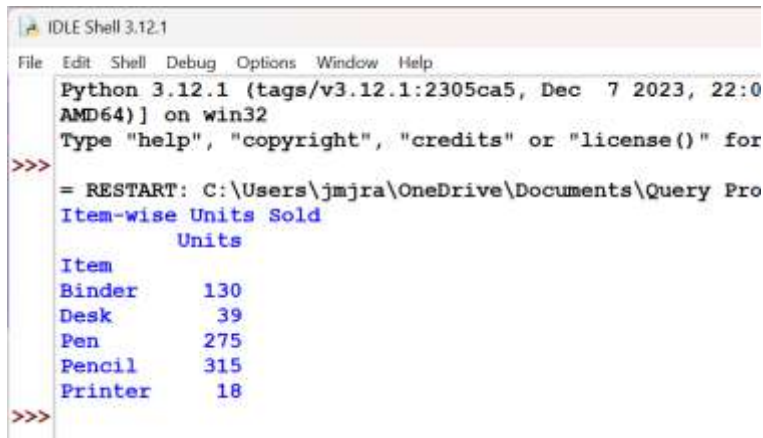
Code:

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
pivot = pd.pivot_table(
    df,
    values="Units",
    index="Item",
    aggfunc="sum"
)
```

```
print("Item-wise Units Sold")
```

```
print(pivot)
```

Output:



```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:0
AMD64) on win32
Type "help", "copyright", "credits" or "license()" for
>>>
= RESTART: C:\Users\jmjra\OneDrive\Documents\Query Pro
Item-wise Units Sold
      Units
Item
Binder    130
Desk       39
Pen       275
Pencil    315
Printer    18
>>>
```

Result:

Item-wise units sold were displayed successfully.

9. Write a Pandas program to create a Pivot table and find the total sale amount region wise, manager wise, sales man wise.

Create a Pivot Table for Region-wise, Manager-wise and Salesman-wise Sales

Aim:

To create a Pivot Table showing total sales region-wise, manager-wise and salesman-wise.

Algorithm:

1. Import the Pandas library.
2. Read the sales_data.csv file.
3. Create a Pivot Table using Region, Manager and SalesMan.
4. Display the total sale amount.

Input CSV File: sales_data.csv

OrderDate,Region,Manager,SalesMan,Item,Units,Unit_price,Sale_amt

1-6-18,East,Martha,Alexander,Television,95,1198,113810

1-23-18,Central,Hermann,Shelli,Home Theater,50,500,25000

2-9-18,Central,Hermann,Luis,Television,36,1198,43128

2-26-18,Central,Timothy,David,Cell Phone,27,225,6075

3-15-18,West,Timothy,Stephen,Television,56,1198,67088

4-1-18,East,Martha,Alexander,Home Theater,60,500,30000

4-18-18,Central,Martha,Steven,Television,75,1198,89850
5-5-18,Central,Hermann,Luis,Television,90,1198,107820
5-22-18,West,Douglas,Michael,Television,32,1198,38336
6-8-18,East,Martha,Alexander,Home Theater,60,500,30000
6-25-18,Central,Hermann,Sigal,Television,90,1198,107820
7-12-18,East,Martha,Diana,Home Theater,29,500,14500
7-29-18,East,Douglas,Karen,Home Theater,81,500,40500
8-15-18,East,Martha,Alexander,Television,35,1198,41930
9-1-18,Central,Douglas,John,Desk,2,125,250
9-18-18,East,Martha,Alexander,Video Games,16,58.5,936
10-5-18,Central,Hermann,Sigal,Home Theater,28,500,14000
10-22-18,East,Martha,Alexander,Cell Phone,64,225,14400

Code:

```
import pandas as pd
df = pd.read_csv("sale_data.csv")
pivot = pd.pivot_table(
    df,
    values="Sale_amt",
    index=["Region", "Manager", "SalesMan"],
    aggfunc="sum"
)
print("Total Sale Amount Region-wise, Manager-wise, SalesMan-wise")
print(pivot)
```

Output:

```
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more in:
>>>
= RESTART: C:\Users\jnjra\OneDrive\Documents\Query Processing
Total Sale Amount Region-wise, Manager-wise, SalesMan-wise
Sale_amt
Region Manager SalesMan
Central Douglas John 250
Hermann Luis 150948
Shelli 25000
Sigal 121820
Martha Steven 89850
Timothy David 6075
East Douglas Karen 40500
Martha Alexander 231076
Diana 14500
West Douglas Michael 38336
Timothy Stephen 67088
>>>
```

Result:

Region-wise, manager-wise and salesman-wise total sales were displayed successfully.

10. Create a dataframe of ten rows, four columns with random values. Write a Pandas program to highlight the negative numbers red and positive numbers black.

Highlight Negative and Positive Numbers in a DataFrame

Aim:

To highlight negative numbers in red and positive numbers in black in a DataFrame.

Algorithm:

1. Import Pandas and NumPy.
2. Create a DataFrame with random values.
3. Apply conditional styling to the DataFrame.
4. Display the styled DataFrame.

Code:

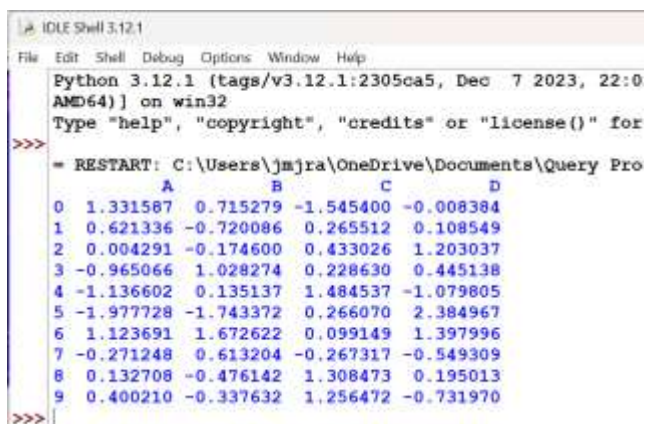
```
import pandas as pd
import numpy as np
np.random.seed(10)
df = pd.DataFrame(
    np.random.randn(10, 4),
    columns=["A", "B", "C", "D"]
)
```



```
def color_numbers(val):
    if val < 0:
        return "color: red"
    else:
        return "color: black"

styled_df = df.style.map(color_numbers)
print(df)
styled_df
```

Output:



```
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:0
AMD64) on win32
Type "help", "copyright", "credits" or "license()" for
>>>
= RESTART: C:\Users\jmjra\OneDrive\Documents\Query Pro
      A      B      C      D
0  1.331587  0.715279 -1.545400 -0.008384
1  0.621336 -0.720086  0.265512  0.108549
2  0.004291 -0.174600  0.433026  1.203037
3 -0.965066  1.028274  0.228630  0.445138
4 -1.136602  0.135137  1.484537 -1.079805
5 -1.977728 -1.743372  0.266070  2.384967
6  1.123691  1.672622  0.099149  1.397996
7 -0.271248  0.613204 -0.267317 -0.549309
8  0.132708 -0.476142  1.308473  0.195013
9  0.400210 -0.337632  1.256472 -0.731970
>>>
```

Result:

Negative values were highlighted in red and positive values in black successfully.